

E-709 SPI Interface

SPI Interface Description

Contents

Using the SPI Interface	2
Definition of Terms.....	2
Overview.....	2
Data Packet Definition.....	3
CRC-16 Generation.....	4
Transport Layer.....	4
Data Segment 1.....	4
Data Segment 2.....	5
Connection Lost	7
Transport Layer Flow Charts.....	8
Nomenclature	8
Sender Flow Charts	9
Receiver Flow Charts.....	11
Application Layer.....	13
Control Value and Actual Value Transfer for the Axis.....	13
OnTarget-Flag Transfer	14
GCS Command Transfer	14
Data Transmission via SPI.....	15
Data Line Definition	15
Packet Transmission.....	16
CS: Data Word Latch	17
SCLK: Serial Data Clock from Host.....	17
DCLK: Data Output Clock.....	17
E-709.1C1L only—LDAT: Latch Data Output / Load Data Input	17
Serial Transmission	18
Physical Layer	18
Transmission Lines	18
Timing Diagrams.....	18
Pin Assignment.....	19
Scaling the Data of Data Segment 1	20
Commands	20
Command Parsers.....	20
GCS Commands	20
Parameter Description	20

Using the SPI Interface

Definition of Terms

PI-controller: E-709, the slave.

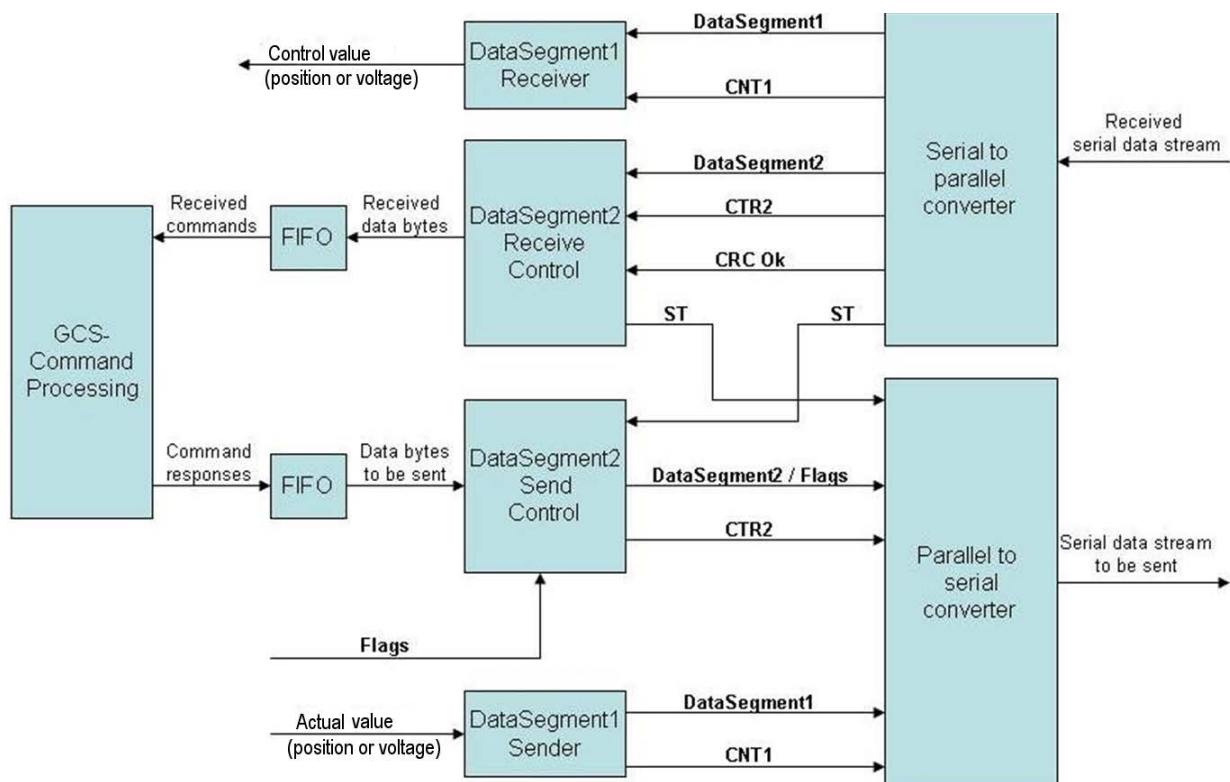
Host: master device which sends commands to the PI-controller.

Overview

The serial data transfer via SPI is primarily designed for transferring actual values from and control values to the PI-controller with minimum latency and update rates as high as the servo frequency of the PI-controller.

It is also possible to send and receive ASCII data so that the host has full access to the PI General Command Set (GCS).

The following block diagram shows the architecture on PI-controller side. The signals shown in this diagram are described in the following sections.



The PI-controller sends actual values to the host and receives new control values from the host. For that purpose data segment 1 is used. The data sent depends on the servo mode selection:

- Closed-loop operation: position values are sent
- Open-loop operation: open-loop target values are sent; the interpretation depends on the PI-controller

For transferring GCS commands data segment 2 is used.

As long as no responses to GCS commands are sent the PI-controller can send flags, for example OnTarget-flags.

All transfers are initiated by the host.

Data Packet Definition

The following data packet definition is valid for both directions.

PID/ST	CTR2/CNT1	DS2/Flags	DS1	CRC-16
1 Byte	1 Byte	2 Bytes	CNT1 x 4 Bytes	2 Bytes

The data frame consists of the following fields:

PID/ST Status bits, explained later

CTR2/CNT1 CTR2 = control bits for data segment 2,
CNT1 = data word counter for data segment 1 (1 word = 4 bytes)

DS2/Flags Data segment 2, for slow data transfer or for flag bits

DS1 Data segment 1, for fast position value transfer

CRC-16 Cyclic redundancy check of PID/ST, CTR2/CNT1, DS2/Flags and DS1

The data frame is for transportation of user data and consist of two data segments DS1 and DS2.

Data segment 1 (DS1) is used to transfer data which can be updated each servo-loop cycle. For the direction from host to PI-controller DS1 is used to transfer control values (axis target position data in closed-loop operation or open-loop target values in open-loop operation). For the direction from PI-controller to host DS1 is used to transfer the actual values (axis position in closed-loop operation; axis position or output voltage in open-loop operation (depending on PI-controller)).

Whenever the PI-controller receives new control values transferred by data segment 1, the PI-controller latches this new data and overwrites the old data. In case that a CRC error occurs and no new control value could be received, the PI-controller uses the latched control value which was most recently received.

Data segment 2 (DS2) is used for slow data transfer, for example for GCS-commands. DS2 transfers fragments of data from transmit fifo to the receiver fifo where data is collected again. Therefore the transfer of a GCS command normally will take several servo-loop cycles.

When data segment 2 transfers no slow data then flag bits are transferred. This is a cyclic transfer like with data segment 1.

CRC-16 Generation

The CRC-16 of PID/ST, CTR2/CNT1, DS2/Flags and DS1 is appended after packet data.

The following polynomial is used to generate the code (Startvalue 0xFFFF): $1 + x^2 + x^{15} + x^{16}$

The CRC-16 word is transferred with the highest bit first and the lowest bit last.

Transport Layer

Data Segment 1

Data segment 1 transports the fast 32-bit data. The number of data words which is currently sent is coded by the CTR2/CNT1 byte:

CTR2				CNT1			
Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
SToggle	TwoBytes	DataCtrl		DS1 Data Word Counter			

The upper part CTR2 (bit 4 through bit 7) of the CTR2/CNT1 byte controls the transmission through data segment 2 and will be explained in the next section.

The lower part CNT1 (bit 0 through bit 3) counts the number N of currently transferred 32-bit data words. The value N can be any number between 0 and 15 so that the position data of up to 15 axes can be transferred.

Note that with E-709, the behavior differs depending on the model:

- E-709.xR,. xRG, .CHG models:
 - In all data packets sent by the slave, the number of axes is 1.
 - Irrespective of the number of axes used by the master, the slave reads only the data for the first axis (i.e. the first data word).
- E-709.1C1L models: The slave sends data for the same number of axes as used by the master in the previous cycle. If more than one axis is used, the second and each further axis responds with 0.

Data Word 1	Data Word 2		Data Word N
4 Byte	4 Byte	4 Byte	4 Byte

The 32-bit data words are sent with the highest bit first and the lowest bit last. The following table shows one 32 bit data word of the data stream:

Data Word i						
b31	B30	B29	b28		b1	b0

Data Segment 2

Data segment 2 is used for slow transfer of GCS commands and their responses. The length of data segment 2 is 2 bytes. When no data is transmitted by data segment 2 then it is used for the cyclic transfer of flag bits. In general a GCS command and its response is longer than 2 bytes and so multiple transfer cycles are needed to transfer a complete command or a command response. In the following, GCS commands or command responses which are transferred by data segment 2 are referred to as “data stream” whereas the part of a GCS command or of a command response which is transferred during one transfer cycle is referred to as “data fraction”.

Both the host and the PI-controller need an additional logic to process data segment 2. In the following, these control units are called sender and receiver. The two transfer directions from host to PI-controller and from PI-controller to host are working completely independent from each other. The architecture of sender and receiver can be the same on host and PI-controller side.

For controlling the transmission via data segment 2 the following control and status bytes are transferred. See also next section for the transport layer flow charts:

CTR2				CNT1			
Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
SToggle	TwoBytes	DataCtrl		DS1 Data Word Counter			

The bits of CTR2 (bit 4 through bit 7) are used by the sender of a data stream. The bits have the following meaning:

DataCtrl (bit 5, bit 4):

(0, 0) = No slow data transferred. When the sender is the PI-controller then the bytes of data segment 2 are used for cyclic flag transfer. The values of bits 6 und 7 (TwoBytes-Flag und SToggle-Flag) are invalid.

(0, 1) = Initialize stream transmission on receiver side. SToggle-Flag is transferred so that the RToggle-Flag on receiver has a definite initial value. No slow data transferred. The bytes of data segment 2 are used for cyclic flag transfer.

(1, 0) = A data fraction of the stream is transmitted by data segment 2. The TwoBytes-Flag und SToggle-Flag are valid and must be evaluated by the receiver.

(1, 1) = Last data fraction of a stream is transmitted by data segment 2. The receiver has the possibility to generate an interrupt so that the receiver FIFO can be read out.

TwoBytes (bit 6):

0 = One data byte is transferred by data segment 2. The second byte of data segment 2 is invalid.

1 = Two bytes are transferred by data segment 2.

The TwoBytes-Flag is valid only when bit 5 is "1" (DataCtrl = (1, 0) or DataCtrl = (1, 1)).

SToggle (bit 7):

In principle the SToggle bit is just the lowest bit of a data counter.

The toggle bit is used to show the receiver that a new fraction of data is sent. The sender shows this by inverting the SToggle value each time when a new data fraction is loaded from the sender FIFO to the transmission register. When the receiver detects that the SToggle changed its value compared with the last data segment 2 transfer cycle then it must save the received data.

When the receiver sends an acknowledge it will reflect the very last SToggle value. For this purpose the RToggle bit of the PID/ST byte is used (see description of the PID/ST byte).

The SToggle-Flag is only valid when DataCtrl $\neq$ (0, 0).

For the acknowledge response the lower bits of the PID/ST byte are used.

PID				ST			
Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
0	0	0	1	0	CRCError	RToggle	ACK

The bits of ST (bit 0 through bit 2) are set by the receiver of a data stream. The bits have the following meaning (bit 3 is currently not used and reserved for future applications):

ACK (bit 0):

0 = No data and no stream initialization received through data segment 2, or data processing is not yet finished, or CRC error occurred.

1 = Acknowledge: receiver tells that valid data was received through data segment 2 and has been processed, or a stream initialization has been processed.

When this bit is zero then the RToggle bit has no meaning.

RToggle (bit 1):

When the ACK bit is non-zero then this bit reflects the most recently received SToggle bit. With this the sender can check if the acknowledge response matches the sent data fraction.

When the sender receives an acknowledge and the RToggle bit matches the SToggle bit then the sender transmits the next data fraction.

CRCError (bit 2):

Tells the sender that the last data packet transmission failed. This bit is always set when an CRC error occurred, regardless if data segment 2 of the data packet contained a data stream or not.

The bits of PID (bit 4 through bit 7) are irrelevant with E-709. They are always set as shown above.

Transfer of a fraction of a data stream (maximum 2 bytes):

Only one byte is transferred (DataCtrl $\neq$ (0, 0) and DataCtrl $\neq$ (0, 1) and TwoBytes = 0). In this case only byte 1 of data segment 2 is used whereas the unused higher byte is set zero:

Data segment 2	
0	Byte 1

Two bytes are transferred (DataCtrl $\neq$ (0, 0) and DataCtrl $\neq$ (0, 1) and TwoBytes = 1). Both bytes of data segment 2 are used:

Data segment 2	
Byte 2	Byte 1

When no bytes are transferred (DataCtrl = (0, 0) or DataCtrl = (0, 1)) then the PI-controller uses the two bytes of data segment 2 to send controller flags (OnTarget flags, for example). The PI-controller always sends the maximum possible number of flags. The CNT1 value doesn't influence the number of transferred flags.

Data segment 2					
Bit 15	Bit 14	...	Bit 2	Bit 1	Bit 0
Flag 16	Flag 15	...	Flag 3	Flag 2	Flag 1

The data segment 2 bytes are transferred with the highest bit first and the lowest bit last.

Connection Lost

When the connection is lost during the transfer of a data stream then the transmission must be aborted. For this purpose the number of CRC-errors should be counted which occur during the transmission of a data stream. When the CRC-error counter reaches a threshold then the transmission will be aborted.

It is up to the sender to set the threshold value where it aborts the transmission.

Because the sender tells the start of the transmission of a data stream the receiver will always be synchronized with the sender. So it is not necessary for the receiver to check the connection.

Transport Layer Flow Charts

The following flow charts show the transfer of data streams via data segment 2. It is shown for sender and receiver side separately.

The flow charts do not show the transfer of the flag bits. Note that these bits are always transferred when data segment 2 is not used for data stream transfer.

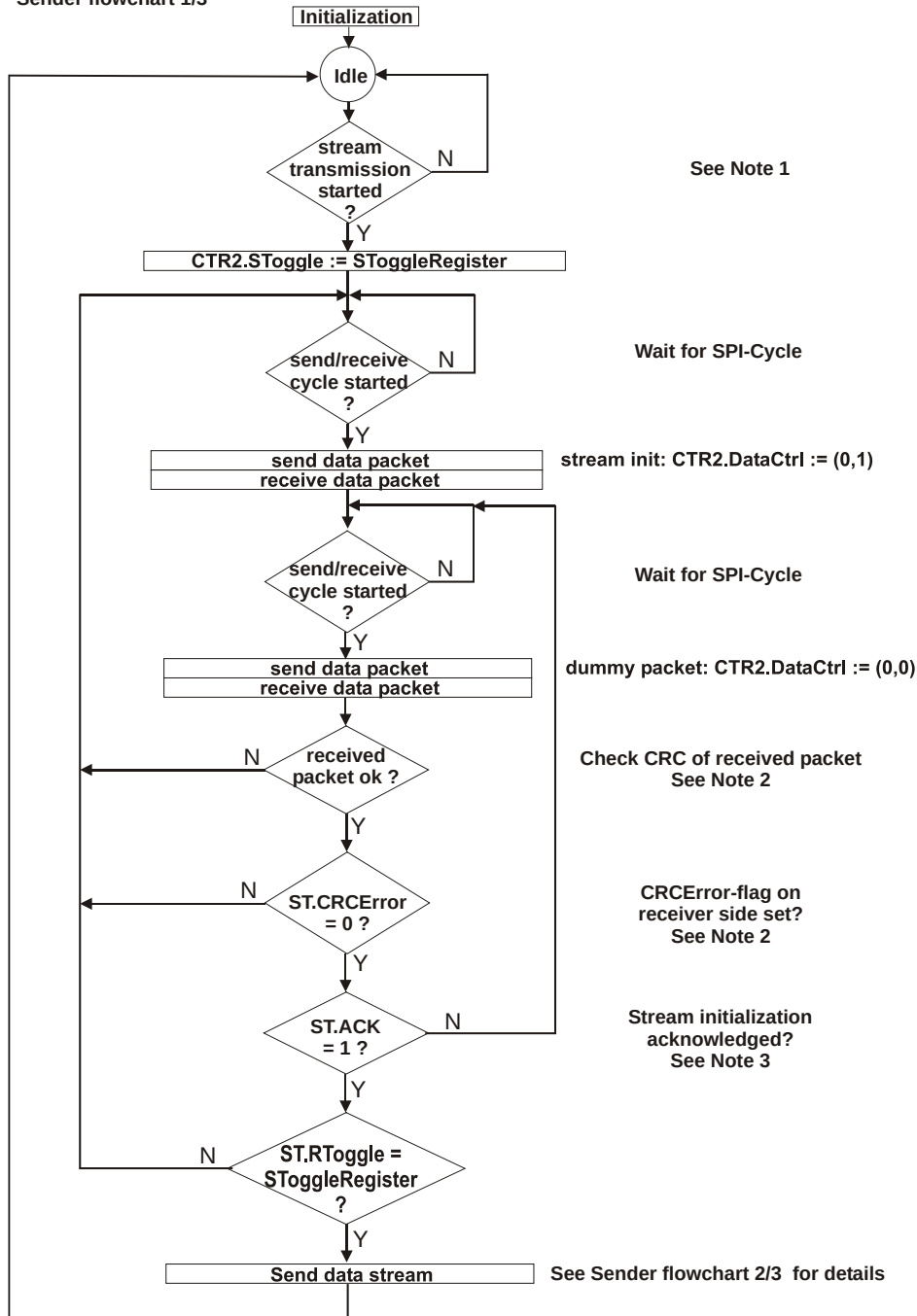
Nomenclature

The following names are used in the flow charts:

- CTR2.SToggle, CTR2.TwoBytes, CTR2.DataCtrl: Control bits for data segment 2 as they are sent with the data packet
- DataCtrlReg: Buffer of DataCtrl bits which are set for data transfer. When data is sent then these bits are copied to CTR2.DataCtrl.
- ST.CRCErrror, ST.RToggle, ST.ACK: Status bits which are set and sent by the receiver.
- SToggleRegister: Toggle register implementation on sender side
- RToggleRegister: Local copy of toggle register on sender side.
- DataSegment2.Byte1, DataSegment2.Byte2: Data bytes transferred by data segment 2. Loading data segment 2 bytes with current flag bits while dummy packets are sent is not shown by the flow charts.

Sender Flow Charts

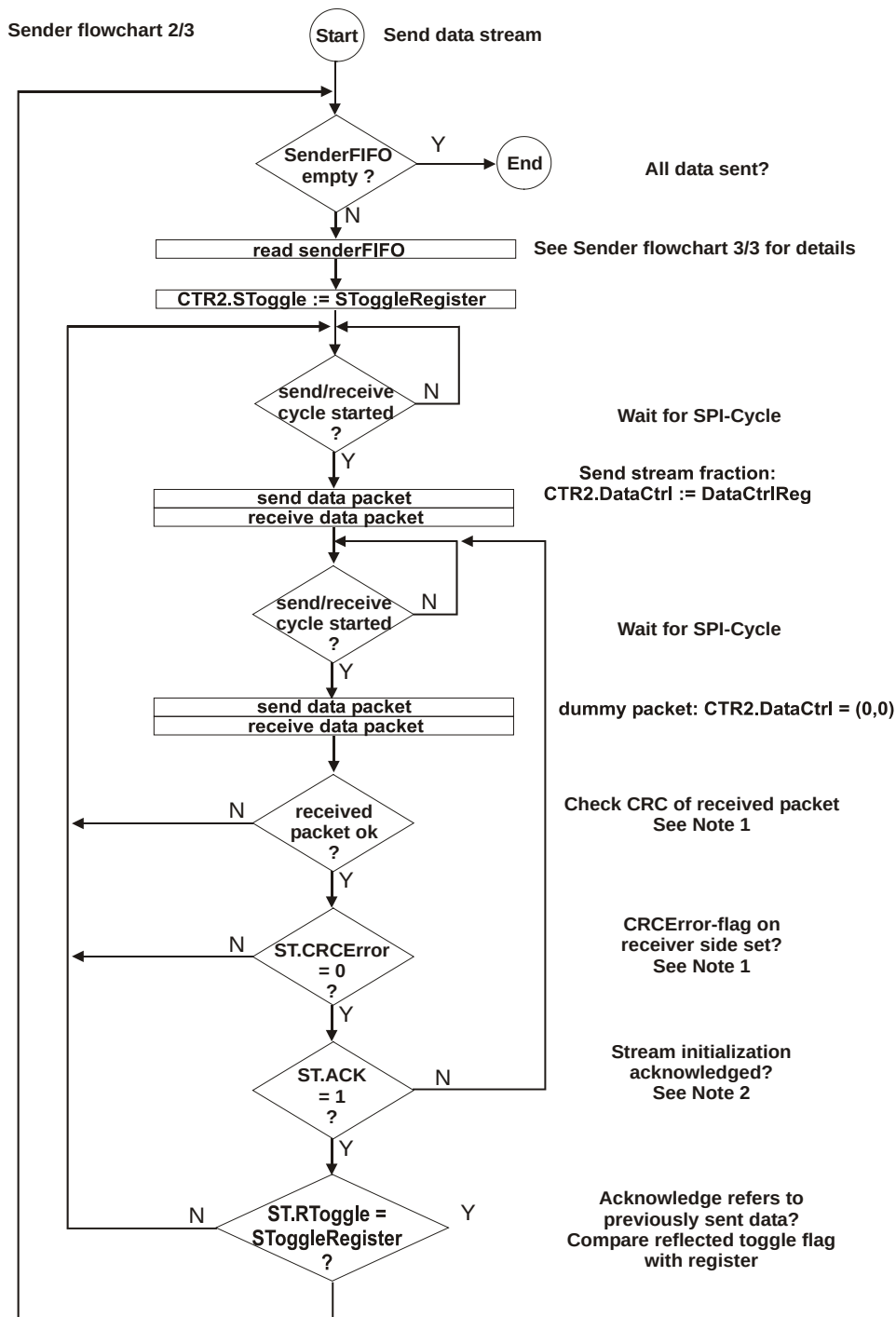
Sender flowchart 1/3

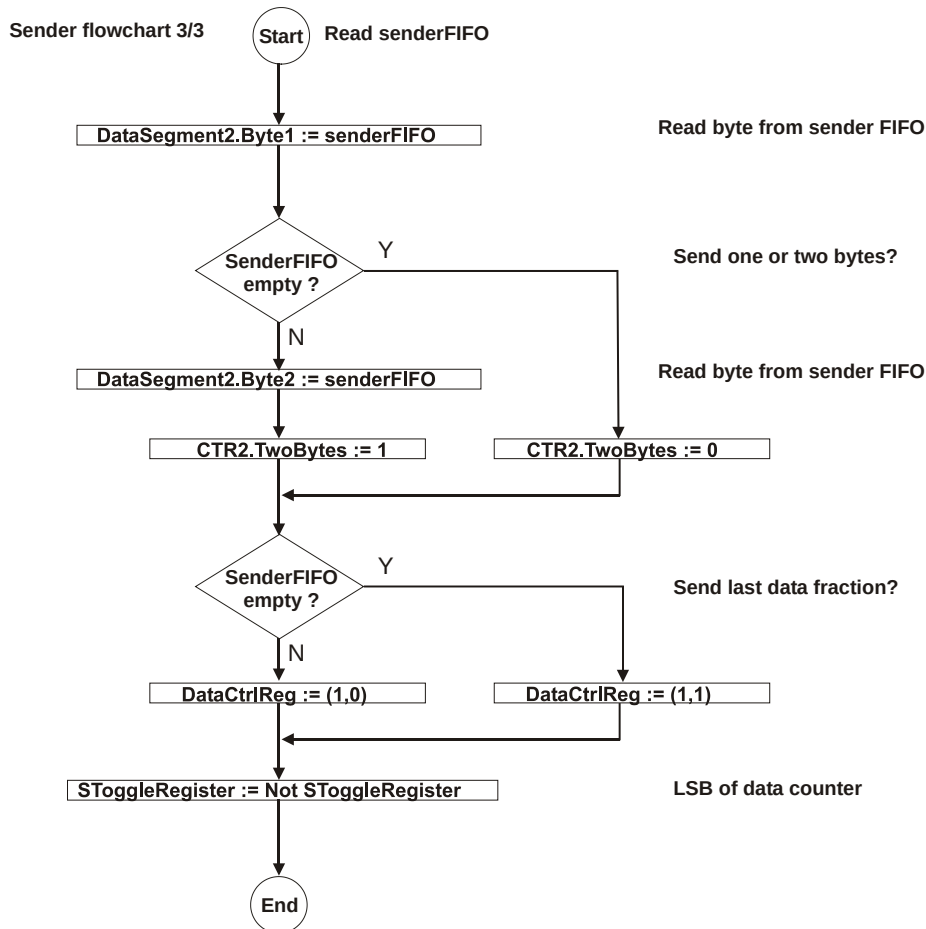


Note 1: Transmission is started after writing the sender FIFO, or when FIFO is full (depends on implementation). Writing of data to sender FIFO is not shown.

Note 2: To check if the connection is lost an error counter can be added. When too many errors occur the state machine returns to the idle state.

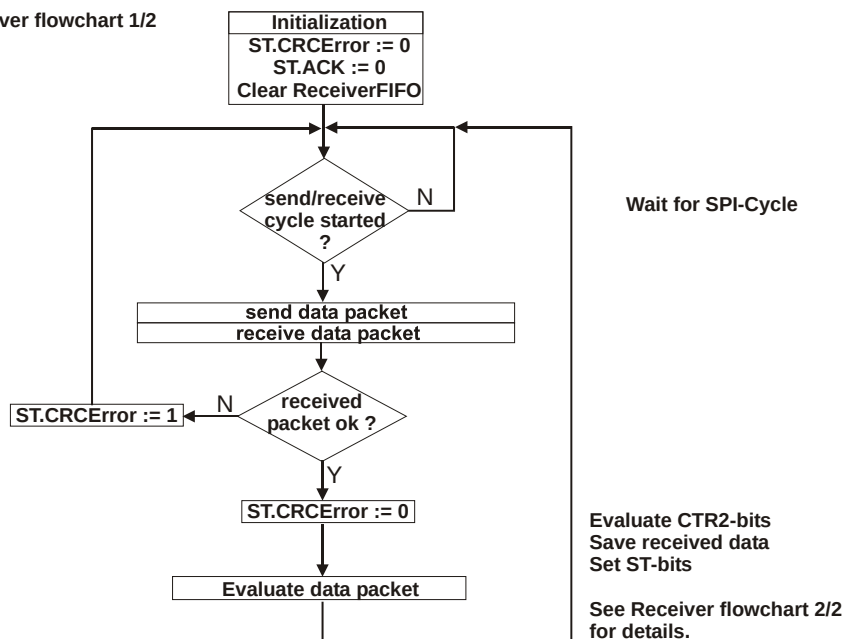
Note 3: Especially for slow systems a time out for acknowledge is useful, because acknowledges could be lost during the system evaluates the received data packets. In this case data should be sent again to force the receiver to send acknowledge again.



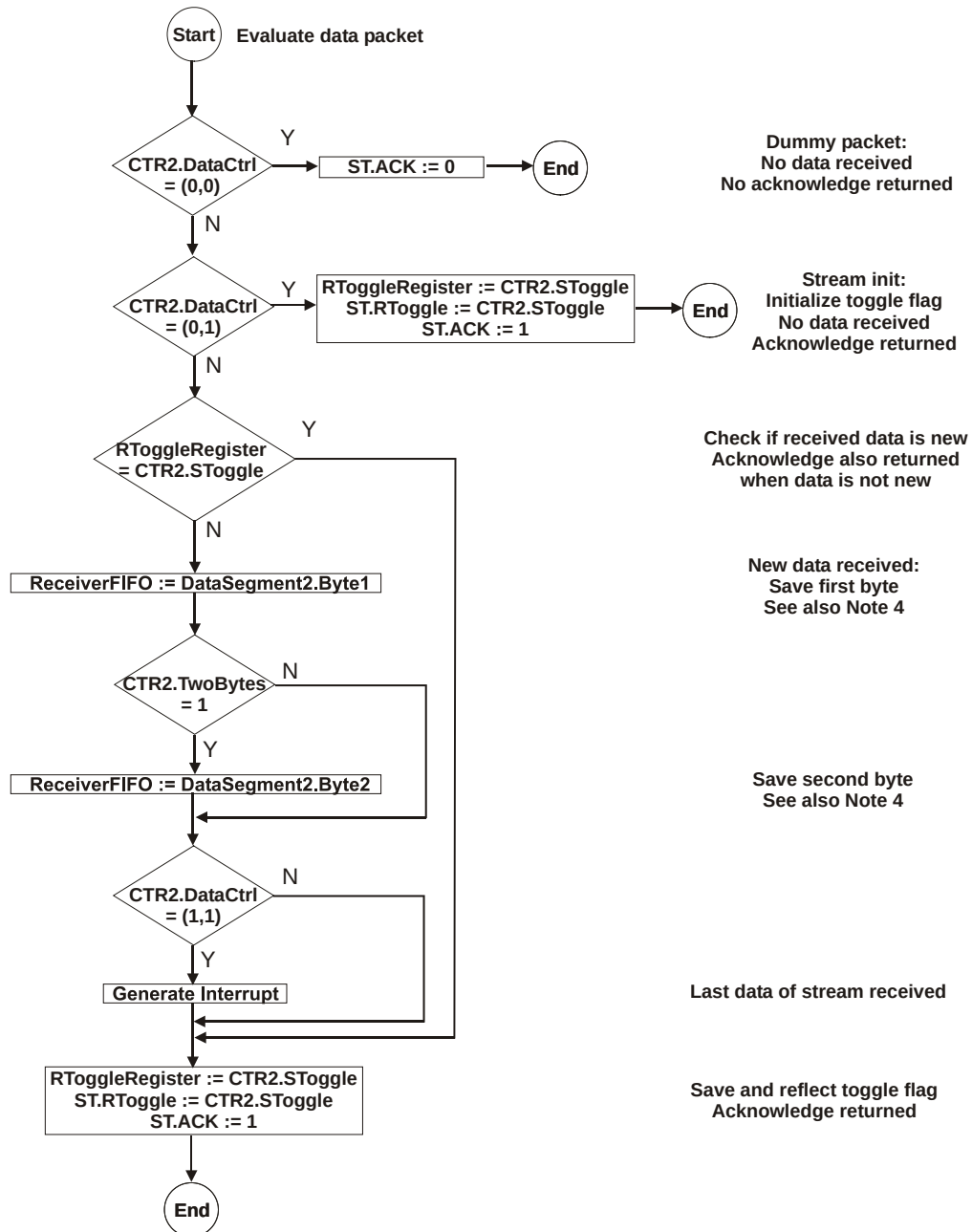


Receiver Flow Charts

Receiver flowchart 1/2



Receiver flowchart 2/2



Note 4: Before writing the received data bytes to FIFO the receiver should check if the FIFO is not full. This is not shown by the flow chart because the implementation might be different, depending on the PI-controller type or host implementation. However, the following procedure is recommended

- Save the received DataSegment2.Byte1, DataSegment2.Byte2 and CTR2 bits temporarily to a buffer.
- Set the ST.ACK bit zero as long as the receiver FIFO is full. This tells the sender that the receiver is not ready to receive any more DataSegment2 data bytes.

- When the FIFO is not full any more save the received data bytes from buffer to the FIFO.
- After saving the data bytes to FIFO the ST.ACK bit should be set to one and the received toggle bit should be returned to the sender (shown at the end of the flow chart above). This tells the sender that the next data fraction can be sent.

Application Layer

In the following, some examples show how the fast axis data and the slow GCS command transfer is realized.

Control Value and Actual Value Transfer for the Axis

Data segment 1 is used for data transfer.

The transfer of control values from host to the PI-controller and the transfer of actual values from the PI-controller to host is done in the same way. Only the interpretation of the values might be different, depending on the controller type (piezo controller, piezo motor controller, dc motor controller...) and the servo mode (closed-loop operation, open-loop operation). In closed-loop operation, axis position data is transferred. In open-loop operation, the interpretation of the values depends on the PI-controller (E-709.xR, .xRG, .CHG: output voltage; E-709.1C1L: axis position).

In any case for each axis 32 bit values are used.

The number of axes which are sent depends on the controller type and on its settings.

When less control values are sent than axes are available in the system then only the first CNT1 axes are updated. For example when the system has 4 axes and the host sends target position data for 3 axes then only axes 1 through 3 are permanently updated by the interface. Axis 4 remains at its initial position.

When the number of control values which are sent is higher than the number of axes which are available then the controller will update the axes which are available and ignore the rest of the received data. For example when the system has 3 axes but the host sends target positions for 4 axes then the controller will update the 3 present axes and ignore the 4th target value.

In the following example data positions for two axes are sent and no GCS command is sent (no data sent through data segment 2). Instead of this servo flags are transferred.

PID/ST	CTR2/ CNT1	DS2	Axis 1				Axis 2				CRC-16	
16	2	Flags	B11	B12	B13	B14	B21	B22	B23	B24	BC1	BC2

where B11 through B14 are the bytes of axis 1 position value, B21 through B24 are the bytes of axis 2 position value and BC1 and BC2 are the CRC-16 bytes.

The highest bits of the data words and the CRC-16 are send first and the lowest bits are send last, that is, bytes B11, B21, and BC1 are the highest bytes and bytes B14, B24, and BC2 are the lowest bytes.

OnTarget-Flag Transfer

As long as data segment 2 is not used for GCS command transfer the two bytes are used to transfer flag bits. When the sender is the PI-Controller then the OnTarget flags are sent:

Data segment 2					
Bit 15	Bit 14	...	Bit 2	Bit 1	Bit 0
0	Axis 15	...	Axis 3	Axis 2	Axis 1

GCS Command Transfer

GCS commands are transferred in ASCII format.

The following example shows the transfer of a command to switch on the servo for of axis 1. The GCS representation is:

SVO 1 1

The representation as ASCII numbers, including spaces and terminating line feed, is:

S	V	O	space	1	space	1	Line feed
83	86	79	32	49	32	49	10

For this command eight characters must be transferred and so four transfer packets are needed. In this example only one axis is transferred in data segment 1 (CNT1 = 1). As initial value for SToggle "0" is assumed.

1st packet:

PID/ST	CTR2/ CNT1	DS2		Axis 1				CRC-16	
16	97	86	83	B11	B12	B13	B14	BC1	BC2

DataCtrl = (1,0)

TwoBytes = 1

SToggle = 0

2nd packet:

PID/S T	CTR2/ CNT1	DS2		Axis 1				CRC-16	
16	225	32	79	B11	B12	B13	B14	BC1	BC2

DataCtrl = (1,0)

TwoBytes = 1

SToggle = 1

3rd packet:

PID/S T	CTR2/ CNT1	DS2		Axis 1				CRC-16	
16	97	32	49	B11	B12	B13	B14	BC1	BC2

DataCtrl = (1,0)

TwoBytes = 1

SToggle = 0

4th packet:

PID/S T	CTR2/ CNT1	DS2		Axis 1				CRC-16	
16	241	10	49	B11	B12	B13	B14	BC1	BC2

DataCtrl = (1,1)

TwoBytes = 1

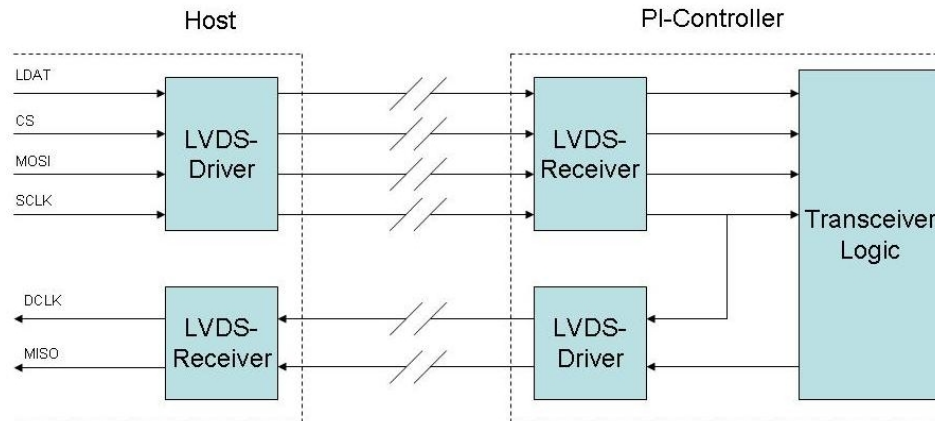
SToggle = 1

Data Transmission via SPI

Data Line Definition

Name	Description	Host	PI-Controller
SCLK	Serial data clock from host	Output	Input
CS	Data word latch. E-709.1C1L only, optional: Frame synchronization	Output	Input
MISO	Master Input Slave Output Data line. Data output changes with falling edge of SCLK	Input	Output
MOSI	Master Output Slave Input Data line. Data input is latched with rising edge of SCLK	Output	Input
DCLK	Serial data clock from PI-controller	Input	Output
LDAT	E-709.1C1L only: Frame synchronization	Output	Input

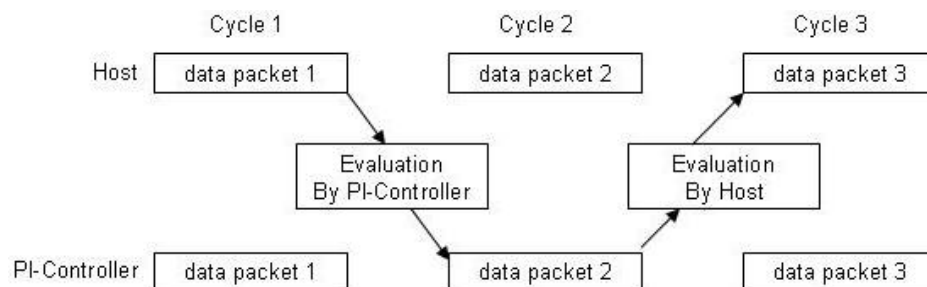
Note that the LDAT signal shown in the figure below is only used with E-709.1C1L models. All other E-709 models do **not** use the LDAT signal.



Packet Transmission

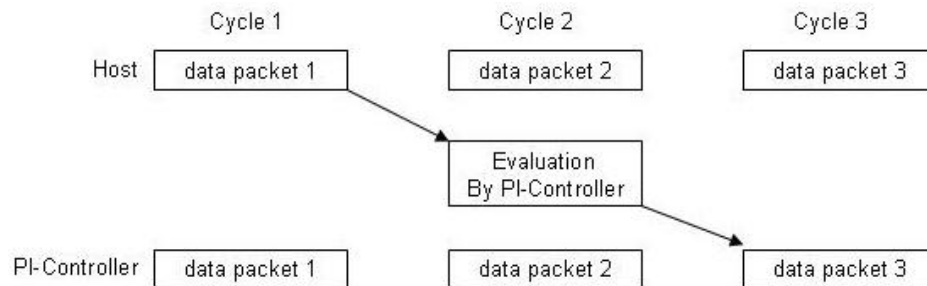
Because the SPI interface has only one clock the data packet transmission of host and PI-controller runs simultaneously. The PI-controller responds at least one cycle after the host command was received. This applies to the GCS commands and their responds which are transferred by data segment 2 and which need to be acknowledged by the receiver.

In the following example, data packet 2 sent by the PI-controller contains the response to the command contained in data packet 1, sent by the host: For example, the host sends the first data fraction with data packet 1, the PI-controller sends an acknowledge to this with data packet 2 and then the host can send the next fraction with data packet 3.



The data packets have the same format as described in the data packet definition section.

It is also possible that the PI-controller needs more than one cycle for processing of data which was sent by the host. In the following example the PI-controller needs one extra cycle for command data processing so that data packet 3 from PI-controller is the answer to data packet 1 from the host side.



CS: Data Word Latch

The CS-signal divides the command packet into data words (word size: at least 16-bit). The CS-signal becomes low while a data word is transferred. With the rising edge of CS the controller internally saves the last data word.

It is also possible to let the CS-signal low during the transfer of one complete frame. In this case the CS-signal behaves identical to the LDAT signal. That's why with E-709.1C1L models, the CS-signal can also be used for frame synchronization, see "E-709.1C1L only—LDAT: Latch Data Output / Load Data Input" below for details.

SCLK: Serial Data Clock from Host

The serial clock is active only while data bits are transferred. The reason for this is that some devices do not have enable inputs for the serial clock.

DCLK: Data Output Clock

The PI-controller returns the SCLK as DCLK line (see next figure). The host can use the DCLK line for the reception of the MISO line. With this method the delays caused by the transmission line, the LVDS-divers and receivers can be compensated. For this reason higher data rates are possible with DCLK.

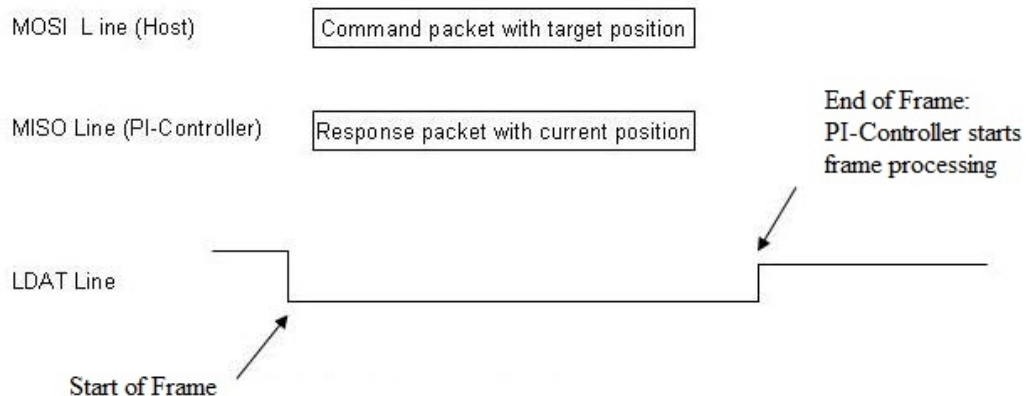
It is possible to run the interface without using the DCLK line. But in this case the delays on the transmission lines and the driver and receiver components become the limiting factor for the transmission rate. For proper data transfer it is therefore recommended to use the DCLK line in every case with data rates higher than 15 MHz.

With E-709.1C1L models, the source selection for the DCLK signal is possible via the Fast IF DCLK Signal Source parameter, ID 0x1000050A as follows:

- Return clock for incoming SCLK signal from master (default); parameter value = 0
- E-709.1C1L servo cycle clock; parameter value = 1

E-709.1C1L only—LDAT: Latch Data Output / Load Data Input

With E-709.1C1L models, the LDAT signal is the frame synchronization signal by default. Falling edge of LDAT signal shows the start of each SPI frame and the rising edge of LDAT signal shows the end of each SPI frame. PI-controller will start internal frame processing after each rising edge of LDAT signal, the recently received data is loaded into the target register of the controller and the axis value is latched for next data frame transmission. In the figure below, the axis values are interpreted as positions.



Optionally the CS-signal can be used instead of the LDAT signal for frame synchronization. The selection is made via the Fast IF LDAT Signal Source parameter, ID 0x1000050B (0 = LDAT signal (default), 1 = CS-signal). If used for frame synchronization, it is essential that the CS-signal is active over the whole frame.

The frame synchronization signal selected by parameter 0x1000050B can also be used to synchronize the position servo cycles of several E-709.1C1L controllers. For further information, see "3.8.8 Synchronization of Multiple E-709.1C1L" in the E-709.1C1L user manual (PZ305EN).

Serial Transmission

CS is set low before any new word is transferred and either set high when a word transfer is completed or at the latest after one complete frame was transmitted.

All bytes of a packet are transferred serially with the highest bit first. The data bits of the MISO line change with the falling edge of SCLK. The data bits of the MOSI line are latched with the rising edge of SCLK. The SCLK line is active only while data bits are transferred.

Physical Layer

Transmission Lines

Each signal is transmitted by a differential data line pair.

The electrical level is LVDS. On receiver side each line must be terminated with 100 ohm.

Timing Diagrams

Note that the time values depend on controller type. The time values listed below were measured at PI with the E-709 controller:

Note that the servo cycle time of the E-709 is 100 μ s. Make sure that the time period between sending data packets is long enough. Otherwise CRC errors will occur during data transmission.

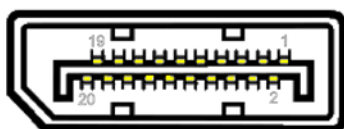
Description	Min	Max
SCLK high time	10ns	
SCLK low time	10ns	

Description	Min	Max
SCLK period time	200ns	
Input setup time: MOSI-bit valid before rising edge of SCLK		35ns
Output setup time: MISO-bit valid after falling edge of SCLK		35ns

Pin Assignment

Note: Currently the LDAT line is only used by the E-709.1C1L. See p. 17 and “3.8.8 Synchronization of Multiple E-709.1C1L” in the E-709.1C1L user manual (PZ305EN).

Connector type: DisplayPort (VESA), lockable



PI-controller (slave):

Pin	Signal Name	Direction
20	N.C.	
19	LDAT-	Input
18	LDAT+	Input
17	CS-	Input
16	GND	
15	CS+	Input
14	GND	
13	GND	
12	SCLK+	Input
11	GND	
10	SCLK-	Input
9	MOSI+	Input
8	GND	
7	MOSI-	Input
6	MISO+	Output
5	GND	
4	MISO-	Output
3	DCLK+	Output
2	GND	
1	DCLK-	Output

Host (master):

Pin	Signal Name	Direction
20	N.C.	
19	LDAT-	Output
18	LDAT+	Output
17	CS-	Output
16	GND	
15	CS+	Output
14	GND	
13	GND	
12	DCLK-	Input
11	GND	
10	DCLK+	Input
9	MISO-	Input
8	GND	
7	MISO+	Input
6	MOSI-	Output
5	GND	
4	MOSI+	Output
3	SCLK-	Output
2	GND	
1	SCLK+	Output

Scaling the Data of Data Segment 1

The Fast IF data limits (parameters 0x10000502 and 0x10000503) are used for scaling the data sent via data segment 1 of the SPI interface:

See the parameter descriptions beginning on p. 20 for further details.

Commands

Command Parsers

The E-709 provides two command parsers and hence two command sets:

- **Fast Command Parser**
Limited number of commands for:
Reading of actual values (position or voltage) and on-target flags
Writing of target positions (closed-loop) or open-loop target values (open-loop)
Only accessible via data segment 1 of the SPI interface.
- **GCS-Parser**
Supports the PI General Command Set (GCS) which provides commands for all controller functionality and controller configuration.
Accessible via the USB or RS-232 interface and via data segment 2 of the SPI interface.

GCS Commands

The GCS commands provided by the firmware of the PI-controller are described in the PI-controller's user manual (PZ305EN for E-709.1C1L; PZ222E for all other E-709 models).

Parameter Description

The parameters listed below are provided by the E-709 firmware in addition to the parameters described in the PI-controller's user manual (PZ305EN for E-709.1C1L; PZ222E for all other E-709 models). Parameter values can be changed using the SPA and SEP commands. To have write access to the parameter, it might be necessary to switch to a higher command level using the CCL command. See the PI-controller's user manual for more information on parameter handling.

Parameter ID (hexa-decimal)	Command Level for write access	Item Type Concerned	Data Type	Parameter Description	Notes
0x10000500	0	Logical Axis	INT	Fast IF Axis Input Usage	Determines the usage of the input values sent via data segment 1 of the SPI interface: 0 = off (input is not used) 1 = target (input used as closed-loop or open-loop control value for the axis) Not supported by all E-709 models: 2 = sensor (input used as current axis position)
0x10000501	0	System	INT	Fast IF Data Type	Selects the data type for the data sent via data segment 1 of the SPI interface. When the 32 bit floating point data type is selected then the data can be read and written directly in axis units. In this case no scaling is necessary. When any of the integer data types is selected then scaling is necessary. In this case the lower and upper limit parameters must also be set (Fast IF Data Low Limit 0x10000502, Fast IF Data High Limit 0x010000503). 0 = FLOAT 32 bit 1 = UINT 16 bit 2 = UINT 24 bit 3 = UINT 32 bit
0x10000502	0	Logical Axis	FLOAT	Fast IF Data Low Limit	Lower limit for the data sent via data segment 1 of the SPI interface. Must be set when any integer data type is selected (Fast IF Data Type, 0x10000501) so that the scaling can be calculated internally. When the selected data type is floating point (IEEE 754) then this parameter has no effect.

Parameter ID (hexa-decimal)	Command Level for write access	Item Type Concerned	Data Type	Parameter Description	Notes
0x10000503	0	Logical Axis	FLOAT	Fast IF Data High Limit	Upper limit for the data sent via data segment 1 of the SPI interface. Must be set when any integer data type is selected (Fast IF Data Type, 0x10000501) so that the scaling can be calculated internally. When the selected data type is floating point (IEEE 754) then this parameter has no effect.
0x1000050A	0	System	INT	Fast IF DCLK Signal Source	With E-709.1C1L only Source selection for the DCLK signal: 0 = SCLK Return (default) = Return clock for incoming SCLK signal from master 1 = Servo CLK = E-709.1C1L servo cycle clock
0x1000050B	0	System	INT	Fast IF LDAT Signal Source	With E-709.1C1L only Source selection for the synchronization signal (p. 17): 0 = HW LDAT (default) = LDAT signal 1 = HW CS = Data word latch signal